

Towards Secure Data Management using Multi-Cryptographic Solutions (Invited)

Shufan Zhang* Xi He* Ashish Kundu† Sujaya Maiyya* Sharad Mehrotra‡ Shantanu Sharma§

*University of Waterloo †Cisco Research ‡University of California, Irvine §New Jersey Institute of Technology
{shufan.zhang,xi.he,smaiyya}@uwaterloo.ca, ashkundu@cisco.com, sharad@ics.uci.edu, shantanu.sharma@njit.edu

Abstract—Several secure data outsourcing systems incorporate various cryptographic techniques to balance security, functionalities, and efficiency. However, their security properties can be ad hoc and sometimes obscure. Our recent work, Secure Normal Form (SNF) [ICDE’24], presents a principled approach that allows data owners to define acceptable leakages of non-sensitive aspects of their data. This approach enables efficient processing of queries while ensuring no unintended leakage of sensitive information. In this paper, we discuss the benefits and challenges of implementing SNF within advanced computational environments and modern data management architectures. We argue that its applicability may extend beyond merely offloading secure query execution to the cloud.

I. INTRODUCTION

As cloud computing rapidly expands with multi-cloud adoption, AI integration, and edge computing, the increasing volume and sensitivity of data processed in diverse and potentially untrusted environments make data encryption essential to protect sensitive data. Researchers have invented fully homomorphic encryption (FHE) and other strong encryption techniques like multi-party computation (MPC), so generic computation can be performed over the encrypted representation. However, these techniques are prohibitively slow, even with the specially-designed hardware accelerator [1], and require a re-implementation of existing software stacks [2].

Trusted execution environments (TEEs), such as Intel SGX [3], have emerged as an alternative hardware-oriented solution for confidential cloud computing. Supported by specialized, isolated memory with hardware encryption, application code can process sensitive data inside TEE, even when the operating system is compromised. Though the state-of-the-art TEE implementation, e.g., SGXv2, allows up to 512 GB of trusted memory and close-to-native execution performance, the performance boost-up only applies to the cases when all the data, intermediate computation results, and outputs fit into the trusted memory. In addition, TEEs are vulnerable to micro-architectural level side-channel attacks [4] and leakages such as access pattern and volume [5]. Defending against these attacks often requires additional protections, such as oblivious execution in between trusted and untrusted memories, which enforces an additional layer of cost to the TEE-based solutions. For example, a single multi-way join query with proper padding may require 10^6 seconds on average [6]. So while strong encryption and provably secure execution exploring FHE and/or TEE, which prevent all types of leakages (e.g., access pattern leakage, volume leakage), may be desirable, such solutions are far from practical today.

Interestingly, secure data outsourcing literature has long realized (and explored) several opportunities to make secure data processing efficient. Such opportunities range from exploring tradeoffs that naturally arise between security, performance, and functionality or by exploiting specificity of the use case in which the secure solution is to be applied. As an example, not every data in a domain may be sensitive and needs protection – depending upon the domain and context, up to 50%-95% of data could be public and non-sensitive [7]. This includes, for example, attributes like product categories, transaction types for product/transaction data, weather conditions, time of the day for contextual data, etc. Such a use case can admit solutions that outsource and process (most of the) non-sensitive data in plaintext, bringing efficiency. Of course, such mixed processing has to be done carefully to prevent leakage [7]. As another example, even when data is sensitive, certain aspects of the data might not be. For example, the ordering of data values or their equality relationships, in itself, might not be sensitive. Such opportunities have also led to significant innovation in secure data processing (e.g., weaker encryption schemes such as order-preserving encryption (OPE) and deterministic encryption (DET) that may leak information about certain properties of data (e.g., ordering/equality) but enable efficient processing such as range queries. Such innovations have led to prior systems (e.g., CryptDB [8], Monomi [9], Jana [10]) that explored a multi-cryptographic approaches to empower users to selectively choose different encryption techniques for different data based on the sensitivity of the data, and expected query workloads. Such systems aim to support trade-offs between the user’s desirable level of security, functionality (e.g., what processing can be done on data in encrypted form), and performance.

The security properties offered by systems supporting multiple cryptographic approaches, however, can be *ad hoc* and sometimes obscure. Take as an example, patient data, where we store weight and blood pressure in addition to disease diagnosis. Since disease may be deemed sensitive, a user may encrypt it using a strong encryption technique while the vital signs/measures are stored using weaker encryption – perhaps, OPE to facilitate range queries. Since measures such as blood pressure and weight have a positive correlation to certain diseases (e.g., diabetes and/or heart disease), the OPE representation of biometrics weakens the security guarantee offered by the strong encryption of the disease attribute. Thus, the actual leakage that occurs in a system when different parts

of the data are stored using different cryptographic primitives may exceed what the data owner has anticipated/expected when choosing a weaker encryption technique for parts of the data [11], [12].

Our recent work on Secure Normal Form (SNF) [13] has addressed the limitation of today's systems when using multiple cryptographic techniques by developing a framework to explore and reason about leakages when using multiple cryptographic techniques in a principled way. A system based on SNF empowers users to first express permissive leakages, such leakages are specified in the form of semantic properties of data using a structured language. The model supports a logic-based framework that enables reasoning about leakages in data based on its semantics. In the running example of patient data, it will enable us to determine that leakage of the ordering in weight and blood pressure, will likely influence the posterior belief the adversary may have about the nature of the disease the patient has. Inspired by the relational database normalization theory¹, we develop mechanisms to support a decomposed representation of data, such that the representation does not have any unintended leakages that the user has not explicitly allowed. While several SNF forms of data representations may exist (e.g., encrypting all data in strongly encrypted form to prevent any leakage), in SNF we explore diverse data partitioning techniques to ensure minimal overhead to query processing. Lastly, SNF is designed to seamlessly integrate with query execution methods, enabling queries and applications to operate efficiently over multi-cryptographic data representations.

In the remainder of this paper, we review SNF's key design choices and explore its potential applications beyond traditional row-based database management systems. These applications include modern data management architectures optimized for OLAP queries (e.g., columnar storage), general programs, and multi-server environments with varying trust boundaries. We discuss the unique opportunities and challenges of implementing SNF in these settings.

II. SECURE NORMAL FORM: MITIGATING UNINTENDED LEAKAGES IN ENCRYPTED DATABASES

The SNF representation is designed to eliminate unintended leakages in multi-cryptographic systems when data is stored at rest. It is tailored for row-oriented relational databases, where data is structured as tuples containing one or more attributes. SNF assumes that the outsourced data resides in either a single relation or multiple uncorrelated relations. To construct an SNF representation, the original database is partitioned based on data semantics and the leakage properties of the cryptographic techniques used. In the following, we briefly review the key concepts and innovations of SNF.

Specification of Permissible Leakages. Leakages in SNF are characterized into *static leakages*, which occur at the data representation level, and *dynamic leakages*, which happen when

queries are executed. Static leakages, based on the inherent properties of the data, can be further classified into three fine-grained types: *value leakage*, which reveals a subset of domain values from the encrypted representation, *distribution leakage*, which exposes the plaintext distribution of data, and *relationship leakage*, which leaks the ℓ -ary relationships among subsets of plaintext values through their ciphertext representations. SNF is further orchestrated with a SQL-like domain-specific language for users, or data owners, to specify which of the aforementioned leakages are allowed on which part of the data.

Unintended Leakages. Unintended leakages can happen across data represented by multiple cryptographic techniques with different security guarantees. Since subsets of the data are represented with techniques that allow different permissible leakages (characterized through the value, distribution, and relationship properties), the co-located columns or rows carry information about data semantics, such as dependencies or correlations, to leak across encrypted representations.

Secure Normal Form (SNF). An encrypted data representation is said to be in SNF if the set of leakages inferred from the representation about its plaintext is limited to only those specified by the data owner. Such representation, that partitions the original data representation into multiple sub-relations, should satisfy the following two properties: (1) *sub-relations unlinkability*, which prevents the adversary from linking tuples from two or more sub-relations, and (2) *loss-less reconstructability*, which allows the reconstruction of the original database from the partitioned representations. The first property of SNF effectively ensures unintended leakages at the presentation level cannot occur, and the second property allows queries on multiple sub-relations to be executed over the SNF representation. To achieve these two properties simultaneously, the data owner may partition the correlated attributes into sub-relations and add strongly encrypted tuple-id (or tid) columns (using different encryption keys) to every sub-relation.

Leakage Reasoning and Partitioning. Based on the permissible leakages specified by data owners, an SNF implementation must carefully select encryption techniques that disclose no more than the allowed leakages. Additionally, it must analyze data semantics to detect unintended leakages, which are represented through dependencies and independencies among the subsets of data. To mitigate unintended leakages, the system employs partitioning strategies that separate co-located encrypted data where such leakages are identified. Among various encrypted representations in SNF, maximally permissive SNFs – where adding any attribute or weakening encryption would violate SNF properties – are particularly desirable for efficient query processing. To achieve this, greedy heuristics are proposed for optimizing data partitioning.

III. EXTENDING SNF TO DYNAMIC QUERY EXECUTION

A. Query Execution with SNF

Data is represented in SNF before outsourcing to the cloud and users may want to execute queries over the outsourced

¹Relational design theory helps determine when a relational schema R contains redundancy and supports algorithms to decompose R into sub-relations to prevent redundant representation of data.

database. SNF is tightly integrated with query execution mechanisms to process user queries securely and efficiently. Consider the read-only queries. If the query only contains attributes in a single sub-relation, this query will be matched to this sub-relation and answered directly by exploiting the weak encryption techniques used to represent the data. Otherwise, a query over multiple sub-relations will be rewritten into a query plan that involves the reconstruction of the sub-relations. The reconstruction must be executed without leaking the linkability of tuples among the sub-relations. TEE/SGX can be employed as a vehicle to ensure secure reconstruction, however, access patterns and output volumes may lead to tuple linkability attacks. In such cases, techniques like Oblivious RAM (ORAM) or oblivious query processing with padding mechanisms are used to further protect query execution.

B. Handling Updates with SNF

Consider the following example, which represents a simple user-defined application for smart building IoT systems. The application processes location data to determine room occupancy levels every time someone enters a floor in the building (e.g., using a motion sensor) and collects the set of rooms (i.e., room numbers) within the floor for which the occupancy exceeds a preset threshold. For rooms with high occupancy, the application triggers resource reallocation, such as activating cameras. Suppose this application is executed on a cloud platform. The sensitivity of the data used by the application varies: for instance, the status of motion sensors or cameras may be considered public, while room occupancy levels can be sensitive, as they may indirectly reveal the locations of individuals. For example, if the adversary observes that room i on the third floor was not on the high occupancy list at time t but appears on the list at time $t + 1$, they may correlate this change with external knowledge – such as knowing that Alice entered the third floor at time $t + 1$. This may lead to the inference that Alice is now in room i , thereby leaking her location privacy.

The leakage demonstrated in this example occurs as a consequence of updating occupancy data between two consecutive observations at time t and $t + 1$. Indeed, when occupancy data is updated due to Alice’s entry into the third floor, the newly computed occupancy status includes room i in the high occupancy list. By observing this data update, possibly to a relational database, the adversary is able to infer Alice’s current location. Defending against such attacks requires extending SNF to further capture the data semantics across temporal updates and the contextual correlations through the execution of the program.

IV. SECURE DATA MANAGEMENT BEYOND RELATIONAL DATABASES

A. Towards Columnar Data Representation

Our discussion above has focused on SNF representation in the context of traditional row stores, where records are stored contiguously one after another in a file. The SNF representation partitions records (both horizontally and vertically)

to ensure the security constraints while trying to maximally keep attributes together in the resulting sub-relations. Indeed, SNF may replicate attributes / columns across sub-relations, if security constraints allow for it, in order to preserve the co-location of attributes as much as possible while still keeping storage overheads in control. The key idea is to minimize the number of times we have to reconstruct records based on partial representation across multiple sub-relations in order to answer the queries.

While row stores are typically well suited for traditional OLTP (often write-intensive) workloads and simpler queries, modern databases use columnar storage to support analytical query processing more efficiently. In column stores, such as C-Store [14], which has influenced how data is stored in systems such as Vertica and Redshift, as well as Parquet file format (used in Spark, Azure, Google Cloud), data is instead stored in a columnar representation. As an example, in C-store each table is partitioned vertically into attribute groups called projections, based on how frequently attributes are co-accessed in representative query workloads. Each projection is further partitioned horizontally into segments, sorted by one or more designated sort keys. To support full query functionality, join indices are maintained for each projection, enabling efficient reconstruction of the original table when needed. This design is driven entirely by query performance, offering benefits such as compression, fast columnar scans, late materialization, and SIMD processing, all of which help minimize I/O and CPU costs during query execution.

Addressing SNF representation for columnar stores requires solving the optimization problem

$$\{R_1, \dots, R_n\} = \arg \min_{P_i \in \mathcal{P}} \text{Cost}(W, P_i), \text{ s.t. SNF properties}$$

where P_i is a candidate partition to represent the relations from all possibilities $\mathcal{P}$, W is the expected workload, and $\text{Cost}()$ is the estimated execution cost of W given P_i . The optimization problem will need to be addressed while satisfying the properties of no unintended leakages. Identifying an optimal columnar data representation that adheres to both the leakage specifications and the constraints of SNF opens several questions, e.g., how do we select appropriate sort keys – keys could be chosen to leverage weaker encryption techniques such as OPE, enabling effective segmentation and fast scan-with-filtering in the resulting data representation. However, when sort keys cannot be used – either due to the application of stronger encryption schemes or because their inclusion would violate SNF’s security semantics – an artificial sort key may need to be introduced. This raises new questions about strategically designing and utilizing artificial sort keys to enhance query performance without compromising security guarantees. Addressing these challenges would require rethinking SNF partitioning and execution strategies in a columnar context, offering a promising direction for future work.

B. Towards Semi-Structured Data Management

SNF enables a principled approach to reason about leakages and ensure security when multiple cryptographic techniques

are used to encrypt (different parts of) structured data and to execute SQL queries on the data. The key concept driving SNF is to empower users to explore efficient solutions while ensuring the requisite security properties. Exploiting multi-cryptographic approaches to gain efficiency is not just a concept that applies to structured relational queries and databases. Need for multi-cryptographic approaches, and hence, a notion similar to SNF, may also arise when representing and using other types of data – such as semi-structured or unstructured data, and in other execution environments such as Python, Spark or Hadoop MapReduce. For instance, data from files read into Pandas dataframes may be encrypted with different encryption techniques [15]. While strategies similar to SNF could be applied at the level of representing encrypted dataframes to prevent unintended leakages, the execution mechanisms have to evolve for the operators supported by the more generic computing paradigms beyond SQL queries.

C. Beyond Single Execution Environment

SNF assumes a single cloud server that stores encrypted data and executes queries. However, real-world cloud infrastructures are increasingly hybrid and distributed, involving multiple servers with varying trust assumptions. Some servers may operate within the same administrative domain as the data owner (e.g., within the same company), while others, possibly from external cloud providers, are less trusted. Additionally, the computation may involve trusted proxies from the data owner’s domain and servers with heterogeneous capabilities, such as the presence or absence of TEE/SGX support.

This complex landscape presents key challenges: (1) modeling the trust boundary in a structured and expressive way; (2) extending SNF representations to adapt to hybrid trust models; (3) mitigating cross-trust-boundary leakages, especially those that may arise from computations on non-sensitive data that inadvertently reveal properties of sensitive data. In this setting, SNF’s query execution strategies must evolve. While secure reconstruction using TEE/SGX remains central, new opportunities for optimization arise—such as offloading parts of the computation to trusted proxies or the data owner’s infrastructure. This can reduce reliance on expensive secure hardware and improve performance, while still preserving strong security guarantees across a distributed and trust-diverse computing environment.

D. Extension to Sliced Execution of General Programs

In the ideal case for full security, the user’s entire application should be executed within TEE, with comprehensive protection against known side-channel attacks, such as Spectre, Meltdown, and Rowhammer. This level of protection may be too expensive for some applications, such as running a machine learning model [16]. A more practical way to explore is to split the program based on the sensitivity of variables, and only use TEE (with side-channel protection) for the sensitive part of the execution. It would be interesting to explore if the current principles of SNF designed for database queries can be appropriately extended to such a scenario and reason

about whether additional leakage can happen due to the sliced program execution. We will need to extend SNF to reason about leakages based on not just data semantics but also based on contextual correlations such as user-defined functions and system-level interactions. Developing such a framework would need tools such as taint analysis [17] for fine-grained security analysis and enforcement across execution environments involving both trusted and untrusted components.

V. CONCLUSION

We discuss the multi-cryptographic approach as a viable solution for secure data management in cloud computing and the secure normal forms for preventing unintended leakages to help users make informed choices when choosing encryption to outsource their data. We also envision potential extensions of the multi-cryptographic and SNF approach for other workloads and general programs.

Acknowledgment: This work is supported by NSF Grants No. 2032525, 1545071, 1527536, 1952247, 2008993, 2133391, 2245372, 2245374 and a gift from the Cisco Research.

REFERENCES

- [1] N. Samardzic, A. Feldmann, A. Krastev, S. Devadas, R. Dreslinski, C. Peikert, and D. Sanchez, “F1: A fast and programmable accelerator for fully homomorphic encryption,” in *MICRO 2021*.
- [2] S. Bian, Z. Zhang, H. Pan, R. Mao, Z. Zhao, Y. Jin, and Z. Guan, “HE3DB: an efficient and elastic encrypted database via arithmetic-and-logic fully homomorphic encryption,” in *CCS 2023*.
- [3] B. Fisch, D. Vinayagamurthy, D. Boneh, and S. Gorbunov, “IRON: functional encryption using intel SGX,” in *CCS 2017*.
- [4] D. Wang, Z. Qian, N. Abu-Ghazaleh, and S. V. Krishnamurthy, “Papp: Prefetcher-aware prime and probe side-channel attack,” in *DAC 2019*.
- [5] S. Han, V. Chakraborty, M. T. Goodrich, S. Mehrotra, and S. Sharma, “Veil: A storage and communication efficient volume-hiding algorithm,” *Proc. ACM Manag. Data* 2023.
- [6] Z. Chang, D. Xie, S. Wang, F. Li, and Y. Shen, “Towards practical oblivious join processing,” *TKDE* 2024.
- [7] K. Y. Oktay, M. Kantarcioglu, and S. Mehrotra, “Secure and efficient query processing over hybrid clouds,” in *ICDE 2017*.
- [8] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan, “Cryptdb: protecting confidentiality with encrypted query processing,” in *SOSP 2011*.
- [9] S. Tu, M. F. Kaashoek, S. Madden, and N. Zeldovich, “Processing analytical queries over encrypted data,” *Proc. VLDB Endow.* 2013.
- [10] D. W. Archer, D. Bogdanov, Y. Lindell, L. Kamm, K. Nielsen, J. I. Pagter, N. P. Smart, and R. N. Wright, “From keys to databases - real-world applications of secure multi-party computation,” *Comput. J.* 2018.
- [11] V. Bindshaedler, P. Grubbs, D. Cash, T. Ristenpart, and V. Shmatikov, “The tao of inference in privacy-protected databases,” *Proc. VLDB Endow.* 2018.
- [12] M. Naveed, S. Kamara, and C. V. Wright, “Inference attacks on property-preserving encrypted databases,” in *CCS 2015*.
- [13] S. Zhang, X. He, A. Kundu, S. Mehrotra, and S. Sharma, “Secure normal form: Mediation among cross cryptographic leakages in encrypted databases,” in *ICDE 2024*.
- [14] M. Stonebraker, D. J. Abadi, A. Batkin, *et al.*, “C-store: a column-oriented dbms,” in *VLDB 2005*.
- [15] L. Mingarelli, “Cryptpandas python pypi package,” 2020. <https://pypi.org/project/CryptPandas/>.
- [16] T. Zhou, Y. Luo, S. Ren, and X. Xu, “Nnsplitter: an active defense solution for dnn model via automated weight obfuscation,” in *ICML 2023*.
- [17] H. ElAtali, L. J. Gunn, H. Liljestrand, and N. Asokan, “Blime: Verifiably secure outsourced computation with hardware-enforced taint tracking,” in *NDSS 2024*.